

Figure 12.9. A global pose can be calculated by walking the hierarchy from the joint in question towards the root and model-space origin, concatenating the child-to-parent (local) transforms of each joint as we go.

```
struct SkeletonPose
{
    Skeleton*   m_pSkeleton;    // skeleton + num joints
    JointPose* m_aLocalPose;    // local joint poses
    Matrix44*  m_aGlobalPose;  // global joint poses
};
```

12.4 Clips

In a film, every aspect of each scene is carefully planned out before any animations are created. This includes the movements of every character and prop in the scene, and even the movements of the camera. This means that an entire scene can be animated as one long, contiguous sequence of frames. And characters need not be animated at all whenever they are off-camera.

Game animation is different. A game is an interactive experience, so one cannot predict beforehand how the characters are going to move and behave. The player has full control over his or her character and usually has partial control over the camera as well. Even the decisions of the computer-driven non-player characters are strongly influenced by the unpredictable actions of the human player. As such, game animations are almost never created as long, contiguous sequences of frames. Instead, a game character's movement must be broken down into a large number of fine-grained motions. We call these individual motions *animation clips*, or sometimes just *animations*.

Each clip causes the character to perform a single well-defined action. Some clips are designed to be looped—for example, a walk cycle or run cycle. Others are designed to be played once—for example, throwing an object or tripping and falling to the ground. Some clips affect the entire body of the character—the character jumping into the air for instance. Other clips affect

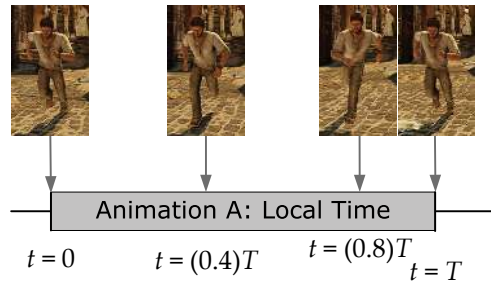


Figure 12.10. The local timeline of an animation showing poses at selected time indices. Images courtesy of Naughty Dog, Inc., © 2014/™ SIE.

only a part of the body—perhaps the character waving his right arm. The movements of any one game character are typically broken down into literally thousands of clips.

The only exception to this rule is when game characters are involved in a noninteractive portion of the game, known as an *in-game cinematic* (IGC), *non-interactive sequence* (NIS) or *full-motion video* (FMV). Noninteractive sequences are typically used to communicate story elements that do not lend themselves well to interactive gameplay, and they are created in much the same way computer-generated films are made (although they often make use of in-game assets like character meshes, skeletons and textures). The terms IGC and NIS typically refer to noninteractive sequences that are rendered in real time by the game engine itself. The term FMV applies to sequences that have been prerendered to an MP4, WMV or other type of movie file and are played back at runtime by the engine’s full-screen movie player.

A variation on this style of animation is a semi-interactive sequence known as a *quick time event* (QTE). In a QTE, the player must hit a button at the right moment during an otherwise noninteractive sequence in order to see the success animation and proceed; otherwise, a failure animation is played, and the player must try again, possibly losing a life or suffering some other consequence as a result.

12.4.1 The Local Timeline

We can think of every animation clip as having a local timeline, usually denoted by the independent variable t . At the start of a clip, $t = 0$, and at the end, $t = T$, where T is the duration of the clip. Each unique value of the variable t is called a *time index*. An example of this is shown in Figure 12.10.

12.4.1.1 Pose Interpolation and Continuous Time

It's important to realize that the rate at which frames are displayed to the viewer is not necessarily the same as the rate at which poses are created by the animator. In both film and game animation, the animator almost never poses the character every $1/30$ or $1/60$ of a second. Instead, the animator generates important poses known as *key poses* or *key frames* at specific times within the clip, and the computer calculates the poses in between via linear or curve-based interpolation. This is illustrated in Figure 12.11.

Because of the animation engine's ability to *interpolate* poses (which we'll explore in depth later in this chapter), we can actually sample the pose of the character at *any time* during the clip—not just on integer frame indices. In other words, an animation clip's timeline is *continuous*. In computer animation, the time variable t is a *real* (floating-point) number, not an *integer*.

Film animation doesn't take full advantage of the continuous nature of the animation timeline, because its frame rate is locked at exactly 24, 30 or 60 frames per second. In film, the viewer sees the characters' poses at frames 1, 2, 3 and so on—there's never any need to find a character's pose on frame 3.7, for example. So in film animation, the animator doesn't pay much (if any) attention to how the character looks in between the integral frame indices.

In contrast, a real-time game's frame rate always varies a little, depending on how much load is currently being placed on the CPU and GPU. Also, game animations are sometimes *time-scaled* in order to make the character appear to move faster or slower than originally animated. So in a real-time game, an animation clip is almost *never* sampled on integer frame numbers. In theory, with a time scale of 1.0, a clip should be sampled at frames 1, 2, 3 and so on. But in practice, the player might actually see frames 1.1, 1.9, 3.2 and so on. And if the time scale is 0.5, then the player might actually see frames

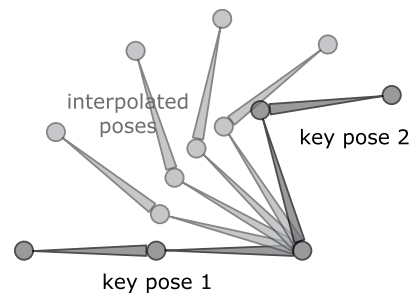


Figure 12.11. An animator creates a relatively small number of key poses, and the engine fills in the rest of the poses via interpolation.

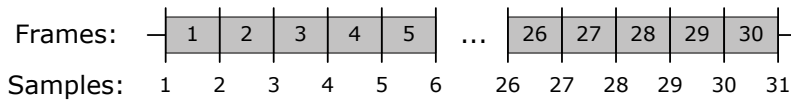


Figure 12.12. A one-second animation sampled at 30 frames per second is 30 frames in duration and consists of 31 samples.

1.1, 1.4, 1.9, 2.6, 3.2 and so on. A negative time scale can even be used to play an animation in reverse. So in game animation, time is both *continuous* and *scalable*.

12.4.1.2 Time Units

Because an animation's timeline is continuous, time is best measured in units of seconds. Time can also be measured in units of *frames*, presuming we define the duration of a frame beforehand. Typical frame durations are $1/30$ or $1/60$ of a second for game animation. However, it's important not to make the mistake of defining your time variable t as an integer that counts whole frames. No matter which time units are selected, t should be a real (floating-point) quantity, a fixed-point number or an integer that measures very small subframe time intervals. The goal is to have sufficient resolution in your time measurements for doing things like "tweening" between frames or scaling an animation's playback speed.

12.4.1.3 Frame versus Sample

Unfortunately, the term *frame* has more than one common meaning in the game industry. This can lead to a great deal of confusion. Sometimes a frame is taken to be a *period of time* that is $1/30$ or $1/60$ of a second in duration. But in other contexts, the term frame is applied to a *single point in time* (e.g., we might speak of the pose of the character "at frame 42").

I personally prefer to use the term *sample* to refer to a single point in time, and I reserve the word *frame* to describe a time period that is $1/30$ or $1/60$ of a second in duration. So for example, a one-second animation created at a rate of 30 frames per second would consist of 31 *samples* and would be 30 *frames* in duration, as shown in Figure 12.12. The term "sample" comes from the field of signal processing. A continuous-time signal (i.e., a function $f(t)$) can be converted into a set of discrete data points by sampling that signal at uniformly spaced time intervals. See Section 14.3.2.1 for more information on sampling.

12.4.1.4 Frames, Samples and Looping Clips

When a clip is designed to be played over and over repeatedly, we say it is *looped*. If we imagine two copies of a 1 s (30-frame/31-sample) clip laid back-to-front, then sample 31 of the first clip will coincide exactly in time with sample 1 of the second clip, as shown in Figure 12.13. For a clip to loop properly, then, we can see that the pose of the character at the end of the clip must exactly match the pose at the beginning. This, in turn, implies that the last sample of a looping clip (in our example, sample 31) is redundant. Many game engines therefore omit the last sample of a looping clip.

This leads us to the following rules governing the number of samples and frames in any animation clip:

- If a clip is *non-looping*, an N -frame animation will have $N + 1$ unique samples.
- If a clip is *looping*, then the last sample is redundant, so an N -frame animation will have N unique samples.

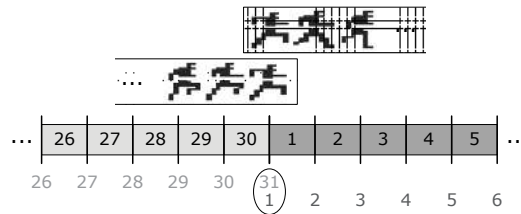


Figure 12.13. The last sample of a looping clip coincides in time with its first sample and is, therefore, redundant.

12.4.1.5 Normalized Time (Phase)

It is sometimes convenient to employ a normalized time unit u , such that $u = 0$ at the start of the animation, and $u = 1$ at the end, no matter what its duration T may be. We sometimes refer to normalized time as the *phase* of the animation clip, because u acts like the phase of a sine wave when the animation is looped. This is illustrated in Figure 12.14.

Normalized time is useful when synchronizing two or more animation clips that are not necessarily of the same absolute duration. For example, we might want to smoothly cross-fade from a 2-second (60-frame) run cycle into a 3-second (90-frame) walk cycle. To make the cross-fade look good, we want to ensure that the two animations remain synchronized at all times, so that the

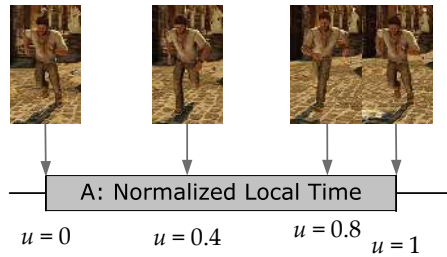


Figure 12.14. An animation clip, showing normalized time units. Images courtesy of Naughty Dog, Inc., © 2014/™ SIE.

feet line up properly in both clips. We can accomplish this by simply setting the normalized start time of the walk clip, u_{walk} , to match the normalized time index of the run clip, u_{run} . We then advance both clips at the same normalized rate so that they remain in sync. This is quite a bit easier and less error-prone than doing the synchronization using the absolute time indices t_{walk} and t_{run} .

12.4.2 The Global Timeline

Just as every animation clip has a local timeline (whose clock starts at 0 at the beginning of the clip), every character in a game has a global timeline (whose clock starts when the character is first spawned into the game world, or perhaps at the start of the level or the entire game). In this book, we'll use the time variable τ to measure global time, so as not to confuse it with the local time variable t .

We can think of *playing* an animation as simply *mapping* that clip's local timeline onto the character's global timeline. For example, Figure 12.15 illustrates playing animation clip A starting at a global time of $\tau_{\text{start}} = 102$ seconds.

As we saw above, playing a looping animation is like laying down an infinite number of back-to-front copies of the clip onto the global timeline. We can also imagine looping an animation a finite number of times, which corresponds to laying down a finite number of copies of the clip. This is illustrated in Figure 12.16.

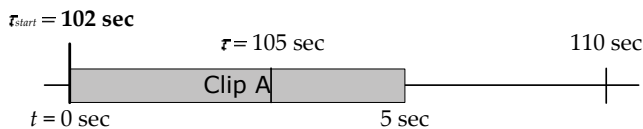


Figure 12.15. Playing animation clip A starting at a global time of 102 seconds.



Figure 12.16. Playing a looping animation corresponds to laying down multiple back-to-back copies of the clip.

Time-scaling a clip makes it appear to play back more quickly or more slowly than originally animated. To accomplish this, we simply scale the image of the clip when it is laid down onto the global timeline. Time-scaling is most naturally expressed as a *playback rate*, which we'll denote R . For example, if an animation is to play back at twice the speed ($R = 2$), then we would scale the clip's local timeline to one-half ($1/R = 0.5$) of its normal length when mapping it onto the global timeline. This is shown in Figure 12.17.

Playing a clip in reverse corresponds to using a time scale of -1 , as shown in Figure 12.18.

In order to map an animation clip onto a global timeline, we need the following pieces of information about the clip:

- its global start time τ_{start} ,

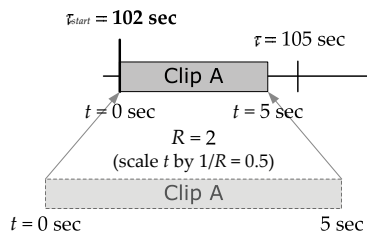


Figure 12.17. Playing an animation at twice the speed corresponds to scaling its local timeline by a factor of $1/2$.

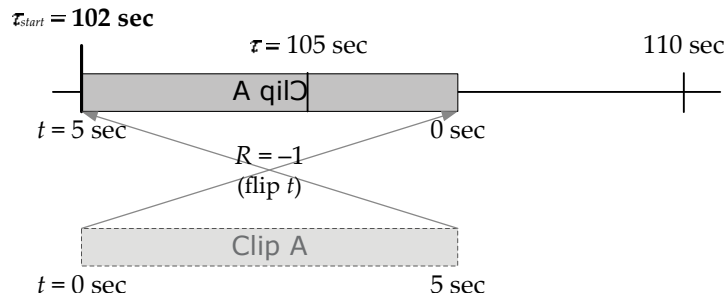


Figure 12.18. Playing a clip in reverse corresponds to a time scale of -1 .

- its playback rate R ,
- its duration T , and
- the number of times it should loop, which we'll denote N .

Given this information, we can map from any global time τ to the corresponding local time t , and vice versa, using the following two relations:

$$\begin{aligned} t &= (\tau - \tau_{\text{start}})R, \\ \tau &= \tau_{\text{start}} + \frac{1}{R}t. \end{aligned} \tag{12.2}$$

If the animation doesn't loop ($N = 1$), then we should clamp t into the valid range $[0, T]$ before using it to sample a pose from the clip:

$$t = \text{clamp} \left[(\tau - \tau_{\text{start}})R \right] \Big|_0^T.$$

If the animation loops forever ($N = \infty$), then we bring t into the valid range by taking the *remainder* of the result after dividing by the duration T . This is accomplished via the *modulo* operator (`mod`, or `%` in C/C++), as shown below:

$$t = ((\tau - \tau_{\text{start}})R) \bmod T.$$

If the clip loops a *finite* number of times ($1 < N < \infty$), we must first clamp t into the range $[0, NT]$ and then modulo *that* result by T in order to bring t into a valid range for sampling the clip:

$$t = \left(\text{clamp} \left[(\tau - \tau_{\text{start}})R \right] \Big|_0^{NT} \right) \bmod T.$$

Most game engines work directly with local animation timelines and don't use the global timeline directly. However, working directly in terms of global times can have some incredibly useful benefits. For one thing, it makes synchronizing animations trivial.

12.4.3 Comparison of Local and Global Clocks

The animation system must keep track of the time indices of every animation that is currently playing. To do so, we have two choices:

- *Local clock.* In this approach, each clip has its own local clock, usually represented by a floating-point time index stored in units of seconds or frames, or in normalized time units (in which case it is often called the *phase* of the animation). At the moment the clip begins to play, the local

time index t is usually taken to be zero. To advance the animations forward in time, we advance the local clocks of each clip individually. If a clip has a non-unit playback rate R , the amount by which its local clock advances must be scaled by R .

- *Global clock.* In this approach, the character has a global clock, usually measured in seconds, and each clip simply records the global time at which it started playing, τ_{start} . The clips' local clocks are *calculated* from this information using Equation (12.2).

The local clock approach has the benefit of being simple, and it is the most obvious choice when designing an animation system. However, the global clock approach has some distinct advantages, especially when it comes to synchronizing animations, either within the context of a single character or across multiple characters in a scene.

12.4.3.1 Synchronizing Animations with a Local Clock

With a local clock approach, we said that the origin of a clip's local timeline ($t = 0$) is usually defined to coincide with the moment at which the clip starts playing. Thus, to synchronize two or more clips, they must be played at exactly the same moment in game time. This seems simple enough, but it can become quite tricky when the commands used to play the animations are coming from disparate engine subsystems.

For example, let's say we want to synchronize the player character's punch animation with a non-player character's corresponding hit reaction animation. The problem is that the player's punch is initiated by the player subsystem in response to detecting that a button was hit on the joy pad. Meanwhile, the non-player character's (NPC) hit reaction animation is played by the artificial intelligence (AI) subsystem. If the AI code runs *before* the player code in the game loop, there will be a one-frame delay between the start of the player's punch and the start of the NPC's reaction. And if the player code runs before the AI code, then the opposite problem occurs when an NPC tries to punch the player. If a message-passing (event) system is used to communicate between the two subsystems, additional delays might be incurred (see Section 16.8 for more details). This problem is illustrated in Figure 12.19.

```
void GameLoop()
{
    while (!quit)
    {
        // preliminary updates...
```

```

    UpdateAllNpcs(); // react to punch event
                      // from last frame

    // more updates...

    UpdatePlayer(); // punch button hit - start punch
                    // anim, and send event to NPC to
                    // react

    // still more updates...
  }
}

```

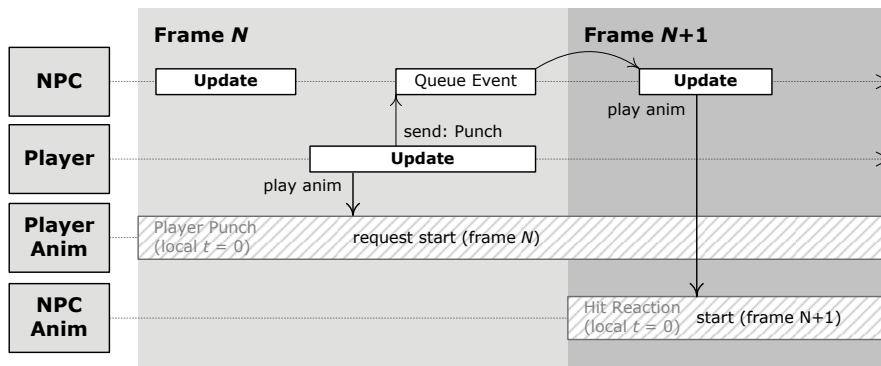


Figure 12.19. The order of execution of disparate gameplay systems can introduce animation synchronization problems when local clocks are used.

12.4.3.2 Synchronizing Animations with a Global Clock

A global clock approach helps to alleviate many of these synchronization problems, because the origin of the timeline ($\tau = 0$) is common across all clips by definition. If two or more animations' global start times are numerically equal, the clips will start in perfect synchronization. If their playback rates are also equal, then they will remain in sync with no drift. It no longer matters *when* the code that plays each animation executes. Even if the AI code that plays the hit reaction ends up running a frame later than the player's punch code, it is still trivial to keep the two clips in sync by simply noting the global start time of the punch and setting the global start time of the reaction animation to match it. This is shown in Figure 12.20.

Of course, we do need to ensure that the two characters' global clocks match, but this is trivial to do. We can either adjust the global start times to

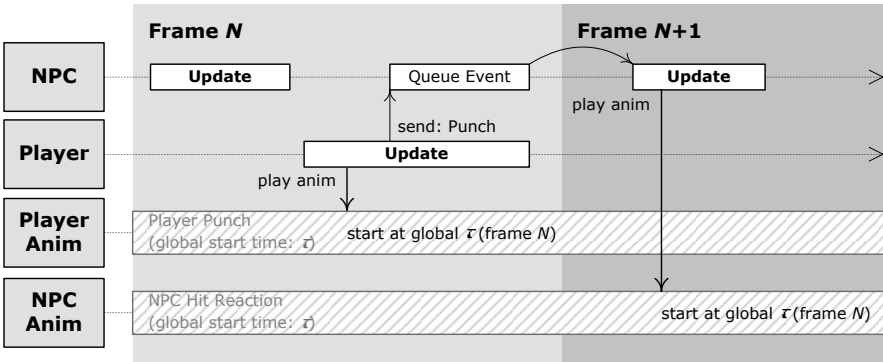


Figure 12.20. A global clock approach can alleviate animation synchronization problems.

take account of any differences in the characters' clocks, or we can simply have all characters in the game share a single master clock.

12.4.4 A Simple Animation Data Format

Typically, animation data is extracted from a Maya scene file by sampling the pose of the skeleton discretely at a rate of 30 or 60 samples per second. A sample comprises a full pose for each joint in the skeleton. The poses are usually stored in SRT format: For each joint j , the scale component is either a single floating-point scalar S_j or a three-element vector $\mathbf{S}_j = [S_{jx} \ S_{jy} \ S_{jz}]$. The rotational component is of course a four-element quaternion $\mathbf{Q}_j = [Q_{jx} \ Q_{jy} \ Q_{jz} \ Q_{jw}]$. And the translational component is a three-element vector $\mathbf{T}_j =$

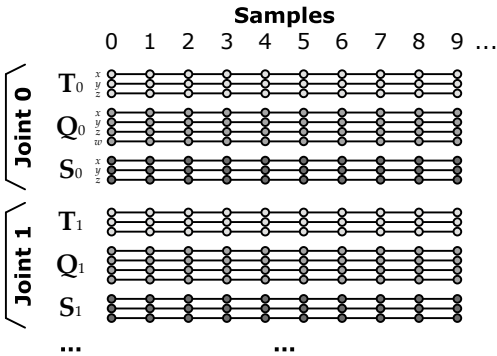


Figure 12.21. An uncompressed animation clip contains 10 channels of floating-point data per sample, per joint.

$[T_{jx} \ T_{jy} \ T_{jz}]$. We sometimes say that an animation consists of up to 10 *channels* per joint, in reference to the 10 components of $\mathbf{S}_j$, $\mathbf{Q}_j$, and $\mathbf{T}_j$. This is illustrated in Figure 12.21.

In C++, an animation clip can be represented in many different ways. Here is one possibility:

```
struct JointPose { ... }; // SRT, defined as above

struct AnimationSample
{
    JointPose*      m_aJointPose; // array of joint
                                // poses
};

struct AnimationClip
{
    Skeleton*       m_pSkeleton;
    F32             m_framesPerSecond;
    U32             m_frameCount;
    AnimationSample* m_aSamples; // array of samples
    bool            m_isLooping;
};
```

An animation clip is authored for a specific skeleton and generally won't work on any other skeleton. As such, our example AnimationClip data structure contains a reference to its skeleton, `m_pSkeleton`. (In a real engine, this might be a unique skeleton id rather than a `Skeleton*` pointer. In this case, the engine would presumably provide a way to quickly and conveniently look up a skeleton by its unique id.)

The number of JointPoses in the `m_aJointPose` array within each sample is presumed to match the number of joints in the skeleton. The number of samples in the `m_aSamples` array is dictated by the frame count and by whether or not the clip is intended to loop. For a non-looping animation, the number of samples is $(m_frameCount + 1)$. However, if the animation loops, then the last sample is identical to the first sample and is usually omitted. In this case, the sample count is equal to `m_frameCount`.

It's important to realize that in a real game engine, animation data isn't actually stored in this simplistic format. As we'll see in Section 12.8, the data is usually *compressed* in various ways to save memory.

12.4.5 Continuous Channel Functions

The samples of an animation clip are really just definitions of continuous functions over time. You can think of these as 10 scalar-valued functions of time

per joint, or as two vector-valued functions and one quaternion-valued function per joint. Theoretically, these *channel functions* are smooth and continuous across the entire clip's local timeline, as shown in Figure 12.22 (with the exception of explicitly authored discontinuities like camera cuts). In practice, however, many game engines interpolate *linearly* between the samples, in which case the functions actually used are *piecewise linear approximations* to the underlying continuous functions. This is depicted in Figure 12.23.

12.4.6 Metachannels

Many games permit additional “metachannels” of data to be defined for an animation. These channels can encode game-specific information that doesn't have to do directly with posing the skeleton but which needs to be synchronized with the animation.

It is quite common to define a special channel that contains *event triggers* at various time indices, as shown in Figure 12.24. Whenever the animation's local time index passes one of these triggers, an *event* is sent to the game engine, which can respond as it sees fit. (We'll discuss events in detail in Chapter 16.) One common use of event triggers is to denote at which points during the

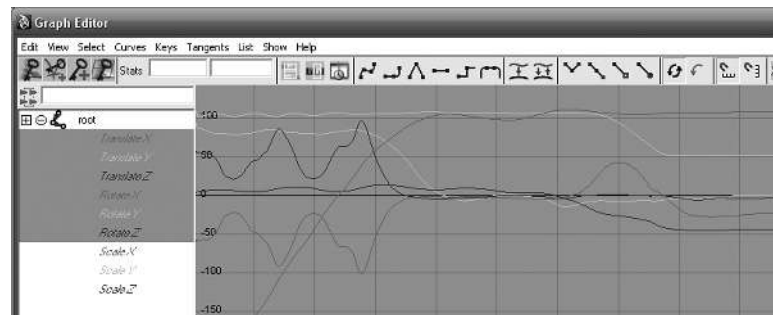


Figure 12.22. The animation samples in a clip define continuous functions over time.

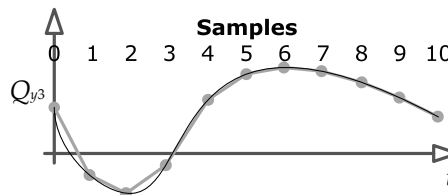


Figure 12.23. Many game engines use a piecewise linear approximation when interpolating channel functions.

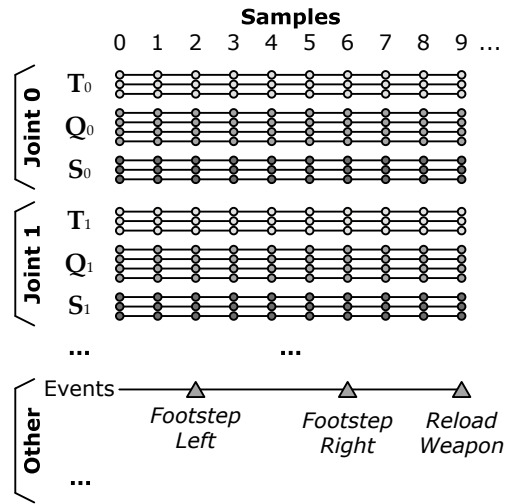


Figure 12.24. A special event trigger channel can be added to an animation clip in order to synchronize sound effects, particle effects and other game events with an animation.

animation certain sound or particle effects should be played. For example, when the left or right foot touches the ground, a footstep sound and a “cloud of dust” particle effect could be initiated.

Another common practice is to permit special joints, known in Maya as *locators*, to be animated along with the joints of the skeleton itself. Because a joint or locator is just an affine transform, these special joints can be used to encode the position and orientation of virtually any object in the game.

A typical application of animated locators is to specify how the game’s camera should be positioned and oriented during an animation. In Maya, a locator is constrained to a camera, and the camera is then animated along with the joints of the character(s) in the scene. The camera’s locator is exported and used in-game to move the game’s camera around during the animation. The field of view (focal length) of the camera, and possibly other camera attributes, can also be animated by placing the relevant data into one or more additional *floating-point channels*.

Other examples of non-joint animation channels include:

- texture coordinate scrolling,
- texture animation (a special case of texture coordinate scrolling in which frames are arranged linearly within a texture, and the texture is scrolled by one complete frame at each iteration),

- animated material parameters (color, specular, transparency, etc.),
- animated lighting parameters (radius, cone angle, intensity, color, etc.), and
- any other parameters that need to change over time and are in some way synchronized with an animation.

12.4.7 Relationship between Meshes, Skeletons and Clips

The UML diagram in Figure 12.25 shows how animation clip data interfaces with the skeletons, poses, meshes and other data in a game engine. Pay particular attention to the *cardinality* and *direction* of the relationships between these classes. The cardinality is shown just beside the tip or tail of the relationship arrow between classes—a one represents a single instance of the class, while an asterisk indicates many instances. For any one type of character, there will be one skeleton, one or more meshes and one or more animation clips. The skeleton is the central unifying element—the skins are attached to the skeleton but don't have any relationship with the animation clips. Likewise, the clips are targeted at a particular skeleton, but they have no “knowledge” of the skin meshes. Figure 12.26 illustrates these relationships.

Game designers often try to reduce the number of unique skeletons in the game to a minimum, because each new skeleton generally requires a whole new set of animation clips. To provide the illusion of many different types of characters, it is usually better to create multiple meshes skinned to the same skeleton when possible, so that all of the characters can share a single set of animations.

12.4.7.1 Animation Retargeting

We said above that an animation is typically only compatible with a single skeleton. This limitation can be overcome via *animation retargeting* techniques.

Retargeting means using an animation authored for one skeleton to animate a different skeleton. If the two skeletons are morphologically identical, retargeting may boil down to a simple matter of joint index remapping. But when the two skeletons don't match exactly, the retargeting problem becomes more complex. At Naughty Dog, the animators define a special pose known as the *retarget pose*. This pose captures the essential differences between the bind poses of the source and target skeletons, allowing the runtime retargeting system to adjust source poses so they will work more naturally on the target character.

Other more-advanced techniques exist for retargeting animations authored for one skeleton so that they work on a different skeleton. For more infor-

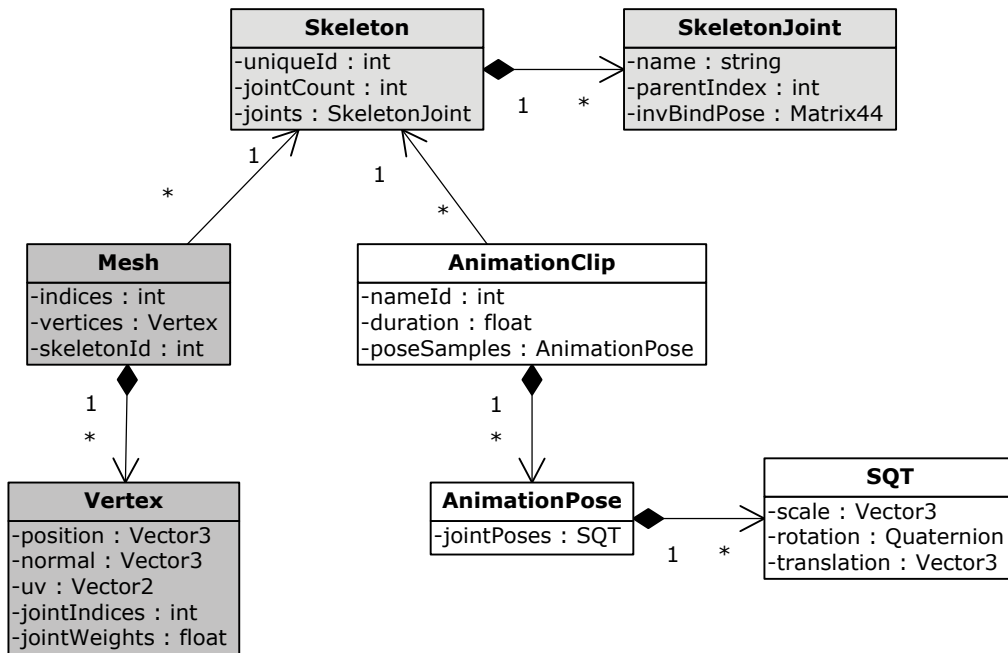


Figure 12.25. UML diagram of shared animation resources.

mation, see “Feature Points Based Facial Animation Retargeting” by Ludovic Dutreuve et al. (<https://bit.ly/2HL9Cdr>) and “Real-time Motion Retargeting to Highly Varied User-Created Morphologies” by Chris Hecker et al. (<https://bit.ly/2vviG3x>).

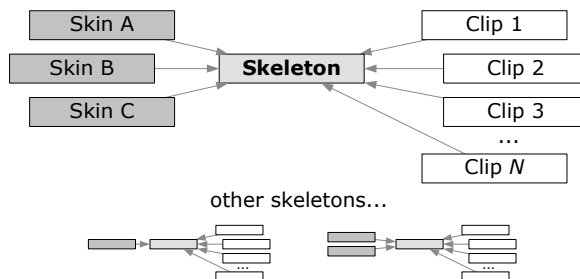


Figure 12.26. Many animation clips and one or more meshes target a single skeleton.

12.5 Skinning and Matrix Palette Generation

We've seen how to pose a skeleton by rotating, translating and possibly scaling its joints. And we know that any skeletal pose can be represented mathematically as a set of local ($\mathbf{P}_{j \rightarrow p(j)}$) or global ($\mathbf{P}_{j \rightarrow M}$) joint pose transformations, one for each joint j . Next, we will explore the process of attaching the vertices of a 3D mesh to a posed skeleton. This process is known as *skinning*.

12.5.1 Per-Vertex Skinning Information

A skinned mesh is attached to a skeleton by means of its vertices. Each vertex can be *bound* to one or more joints. If bound to a single joint, the vertex tracks that joint's movement exactly. If bound to two or more joints, the vertex's position becomes a *weighted average* of the positions it would have assumed had it been bound to each joint independently.

To skin a mesh to a skeleton, a 3D artist must supply the following additional information at each vertex:

- the *index* or *indices* of the joint(s) to which it is bound, and
- for each joint, a *weighting factor* describing how much influence that joint should have on the final vertex position.

The weighting factors are assumed to add to one, as is customary when calculating any weighted average.

Usually a game engine imposes an upper limit on the number of joints to which a single vertex can be bound. A four-joint limit is typical for a number of reasons. First, four 8-bit joint indices can be packed into a 32-bit word, which is convenient. Also, while it's pretty easy to see a difference in quality between a two-, three- and even a four-joint-per-vertex model, most people cannot see a quality difference as the number of joints per vertex is increased beyond four.

Because the joint weights must sum to one, the last weight can be omitted and often is. (It can be calculated at runtime as $w_3 = 1 - (w_0 + w_1 + w_2)$.) As such, a typical skinned vertex data structure might look as follows:

```
struct SkinnedVertex
{
    float  m_position[3];    // (Px, Py, Pz)
    float  m_normal[3];     // (Nx, Ny, Nz)
    float  m_u, m_v;        // texture coords (u,v)
    U8      m_jointIndex[4]; // joint indices
    float  m_jointWeight[3]; // joint weights (last
                           // weight omitted)
};
```

12.5.2 The Mathematics of Skinning

The vertices of a skinned mesh track the movements of the joint(s) to which they are bound. To make this happen mathematically, we would like to find a matrix that can transform the vertices of the mesh from their original positions (in bind pose) into new positions that correspond to the current pose of the skeleton. We shall call such a matrix a *skinning matrix*.

Like all mesh vertices, the position of a skinned vertex is specified in model space. This is true whether its skeleton is in bind pose or in any other pose. So the matrix we seek will transform vertices from model space (bind pose) to model space (current pose). Unlike the other transforms we've seen thus far, such as the model-to-world transform or the world-to-view transform, a skinning matrix is *not* a change of basis transform. It morphs vertices into new positions, but the vertices are in model space both before and after the transformation.

12.5.2.1 Simple Example: One-Jointed Skeleton

Let us derive the basic equation for a skinning matrix. To keep things simple at first, we'll work with a skeleton consisting of a single joint. We therefore have two coordinate spaces to work with: model space, which we'll denote with the subscript M, and the joint space of our one and only joint, which will be indicated by the subscript J. The joint's coordinate axes start out in bind pose, which we'll denote with the superscript B. At any given moment during an animation, the joint's axes move to a new position and orientation in model space—we'll indicate this *current pose* with the superscript C.

Now consider a single vertex that is skinned to our joint. In bind pose, its model-space position is $\mathbf{v}_M^B$. The skinning process calculates the vertex's new model-space position in the current pose, $\mathbf{v}_M^C$. This is illustrated in Figure 12.27.

The “trick” to finding the skinning matrix for a given joint is to realize that the position of a vertex bound to a joint is *constant* when expressed in *that joint's coordinate space*. So we take the bind-pose position of the vertex in model space, convert it into joint space, move the joint into its current pose, and finally convert the vertex back into model space. The net effect of this round trip from model space to joint space and back again is to “morph” the vertex from bind pose into the current pose.

Referring to the illustration in Figure 12.28, let's assume that the coordinates of the vertex $\mathbf{v}_M^B$ are (4, 6) in model space (when the skeleton is in bind pose). We convert this vertex into its equivalent joint-space coordinates $\mathbf{v}_J$, which are roughly (1, 3) as shown in the diagram. Because the vertex is bound

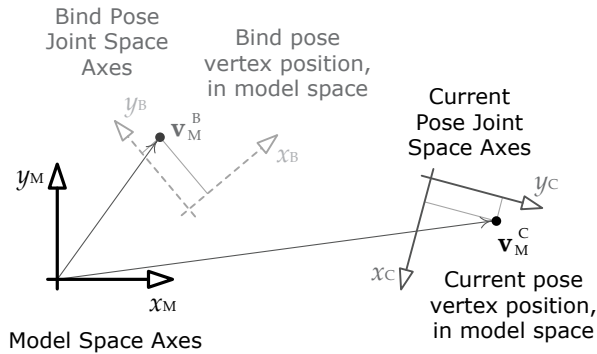


Figure 12.27. Bind pose and current pose of a simple, one-joint skeleton and a single vertex bound to that joint.

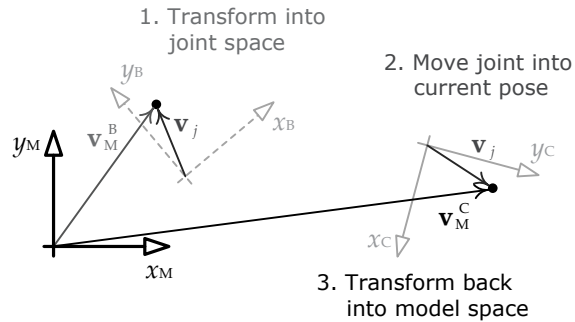


Figure 12.28. By transforming a vertex's position into joint space, it can be made to "track" the joint's movements.

to the joint, its joint-space coordinates will *always* be (1, 3) no matter how the joint may move. Once we have the joint in the desired current pose, we convert the vertex's coordinates back into model space, which we'll denote with the symbol $\mathbf{v}_M^C$. In our diagram, these coordinates are roughly (18, 2). So the skinning transformation has morphed our vertex from (4, 6) to (18, 2) in model space, due entirely to the motion of the joint from its bind pose to the current pose shown in the diagram.

Looking at the problem mathematically, we can denote the *bind pose* of the joint j in model space by the matrix $\mathbf{B}_{j \rightarrow M}$. This matrix transforms a point or vector whose coordinates are expressed in joint j 's space into an equivalent set of model-space coordinates. Now, consider a vertex whose coordinates are expressed in model space with the skeleton in bind pose. To convert these vertex coordinates into the space of joint j , we simply multiply it by the *inverse*

bind pose matrix, $\mathbf{B}_{M \rightarrow j} = (\mathbf{B}_{j \rightarrow M})^{-1}$:

$$\mathbf{v}_j = \mathbf{v}_M^B \mathbf{B}_{M \rightarrow j} = \mathbf{v}_M^B (\mathbf{B}_{j \rightarrow M})^{-1}. \quad (12.3)$$

Likewise, we can denote the joint's *current pose* (i.e., any pose that is *not* bind pose) by the matrix $\mathbf{C}_{j \rightarrow M}$. To convert $\mathbf{v}_j$ from joint space back into model space, we simply multiply it by the current pose matrix as follows:

$$\mathbf{v}_M^C = \mathbf{v}_j \mathbf{C}_{j \rightarrow M}.$$

If we expand $\mathbf{v}_j$ using Equation (12.3), we obtain an equation that takes our vertex directly from its position in bind pose to its position in the current pose:

$$\begin{aligned} \mathbf{v}_M^C &= \mathbf{v}_j \mathbf{C}_{j \rightarrow M} \\ &= \mathbf{v}_M^B (\mathbf{B}_{j \rightarrow M})^{-1} \mathbf{C}_{j \rightarrow M} \\ &= \mathbf{v}_M^B \mathbf{K}_j. \end{aligned} \quad (12.4)$$

The combined matrix $\mathbf{K}_j = (\mathbf{B}_{j \rightarrow M})^{-1} \mathbf{C}_{j \rightarrow M}$ is known as a *skinning matrix*.

12.5.2.2 Extension to Multijointed Skeletons

In the example above, we considered only a single joint. However, the math we derived above actually applies to any joint in any skeleton imaginable, because we formulated everything in terms of global poses (i.e., joint space to model space transforms). To extend the above formulation to a skeleton containing multiple joints, we therefore need to make only two minor adjustments:

1. We must make sure that our $\mathbf{B}_{j \rightarrow M}$ and $\mathbf{C}_{j \rightarrow M}$ matrices are calculated properly for the joint in question, using Equation (12.1). $\mathbf{B}_{j \rightarrow M}$ and $\mathbf{C}_{j \rightarrow M}$ are just the bind pose and current pose equivalents, respectively, of the matrix $\mathbf{P}_{j \rightarrow M}$ used in that equation.
2. We must calculate an array of skinning matrices $\mathbf{K}_j$, one for each joint j . This array is known as a *matrix palette*. The matrix palette is passed to the rendering engine when rendering a skinned mesh. For each vertex, the renderer looks up the appropriate joint's skinning matrix in the palette and uses it to transform the vertex from bind pose into current pose.

We should note here that the current pose matrix $\mathbf{C}_{j \rightarrow M}$ changes every frame as the character assumes different poses over time. However, the inverse bind-pose matrix is constant throughout the entire game, because the bind pose of the skeleton is fixed when the model is created. Therefore, the

matrix $(\mathbf{B}_{j \rightarrow M})^{-1}$ is generally cached with the skeleton, and needn't be calculated at runtime. Animation engines generally calculate local poses for each joint $(\mathbf{C}_{j \rightarrow p(j)})$, then use Equation (12.1) to convert these into global poses $(\mathbf{C}_{j \rightarrow M})$, and finally multiply each global pose by the corresponding cached inverse bind pose matrix $(\mathbf{B}_{j \rightarrow M})^{-1}$ in order to generate a skinning matrix $(\mathbf{K}_j)$ for each joint.

12.5.2.3 Incorporating the Model-to-World Transform

Every vertex must eventually be transformed from model space into world space. Some engines therefore premultiply the palette of skinning matrices by the object's model-to-world transform. This can be a useful optimization, as it saves the rendering engine one matrix multiply per vertex when rendering skinned geometry. (With hundreds of thousands of vertices to process, these savings can really add up!)

To incorporate the model-to-world transform into our skinning matrices, we simply concatenate it to the regular skinning matrix equation, as follows:

$$(\mathbf{K}_j)_W = (\mathbf{B}_{j \rightarrow M})^{-1} \mathbf{C}_{j \rightarrow M} \mathbf{M}_{M \rightarrow W}.$$

Some engines bake the model-to-world transform into the skinning matrices like this, while others don't. The choice is entirely up to the engineering team and is driven by all sorts of factors. For example, one situation in which we would definitely *not* want to do this is when a single animation is being applied to multiple characters simultaneously—a technique known as *animation instancing* that is sometimes used for animating large crowds of characters. In this case we need to keep the model-to-world transforms separate so that we can share a single matrix palette across all characters in the crowd.

12.5.2.4 Skinning a Vertex to Multiple Joints

When a vertex is skinned to more than one joint, we calculate its final position by assuming it is skinned to each joint individually, calculating a model-space position for each joint and then taking a *weighted average* of the resulting positions. The weights are provided by the character rigging artist, and they must always sum to one. (If they do not sum to one, they should be renormalized by the tools pipeline.)

The general formula for a weighted average of N quantities a_0 through a_{N-1} , with weights w_0 through w_{N-1} and with $\sum w_i = 1$ is:

$$a = \sum_{i=0}^{N-1} w_i a_i.$$